Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

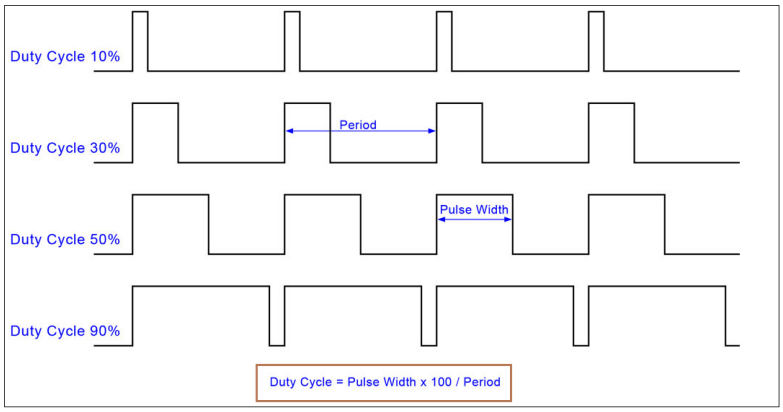
Chair of Computer Science VIII
Aerospace Information Technology
Prof.Dr.Sergio Montenegro.

Exercise:

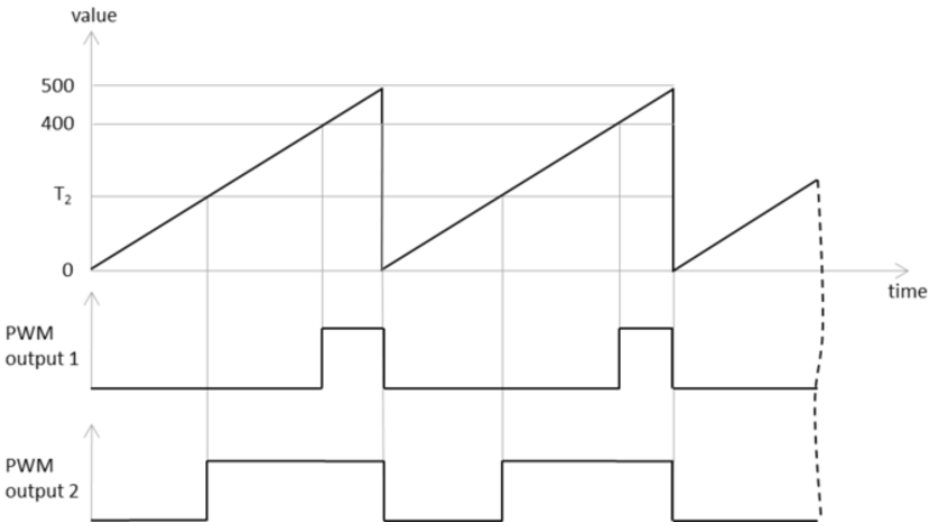
(1) PWM in Stm32f407 for Motor Control
(2)Motor Encoder (Speed Monitoring)

1 Pulse Width Modulation (PWM) using Timer/Counter

Pulse Width Modulation, or PWM, is a technique for getting analog results with digital means. Digital control is used to create a square wave, a signal switched between on and off. This on-off pattern can simulate voltages in between full on (5 Volts) and off (0 Volts) by changing the portion of the time the signal spends on versus the time that the signal spends off. The duration of "on time" is called the pulse width. To get varying analog values, you change, or modulate, that pulse width. If you repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and 5v controlling the brightness of the LED.

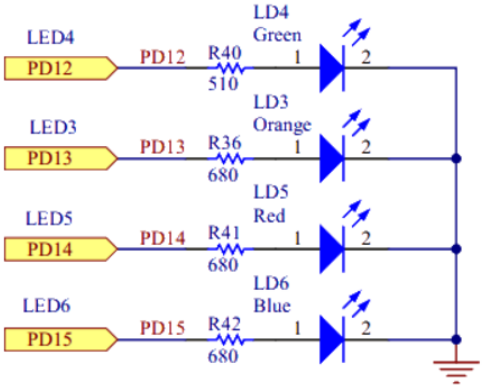


The STM32 hardware timers are separate hardware blocks that can count from 0 to a given value triggering some events in between. In the PWM mode the timer controls the output of 1 or more output channels. When the counter value reaches 0, maximum or a compare value defined for each channel, the output value of the channel can be changed as shown below:



On the STM32F4Discovery board the main LEDs are connected to pins PD12-PD15 having

channels 1-4 of timer 4 as the alternate function 2.



Port		AF0	AF1	AF2
		SYS	TIM1/2	TIM3/4/5
Port D	PD12			TIM4_CH1
	PD13			TIM4_CH2
	PD14			TIM4_CH3
	PD15			TIM4_CH4

1.1 Task1: Controlling the Brigtness of LED using PWM signal

- Generate a PWM signal @ 1KHz duty cycle 25% with timer TIM4, Modify the PWM_Template.c in such a manner that all four channel of TIM4s are connted to the four external LEDs.
- your code should change the brightness of LEDs from 0% to 100%.
- In order to generate a PWM signal using timer, You will have to set the pulse_length parameter in the timer configuration beside the period and prescalar. The pulse_length is set by the following relation:

$$\boxed{Pulse_Length = \frac{(Period + 1) * Duty_Cycle}{100} - 1}$$

- If we want to set PWM duty cycle after initialization ,we can't use TIM_OCInitStruct.TIM_Pulse, but directly write the pulse_length value to TIM4 compare register 1 (CCR1) as shown below:

TIM4 -> CCRx = pulse_length

Note:

- You can use the following delay(uint32_t) function to observe the brightness variation.

1.2 Task2: Motor Control using PWM signal

- Use the motor_pwm.cpp template to control the motor.
- Attach the motor power cable with the DC motor slot.
- Test the motor with Different duty cycles.

1.3 Task3: Motor Encoder (Speed Monitoring)

- Join the power and monitoring wires of the Encoder with the Discovery board.
- Import the motor_Encoder.cpp file in the project.
- Print the motor RPM (rotation per minute) on the screen.